Conosci il tuo dataset

L2 · Informatica per economisti · SIMA 2026

Oggi

1

Spiegazione e demo

25 min

Librerie, DataFrame, metodi e attributi

2

Task

60 min

Esplorare imprese.xlsx, a piccoli passi

3

Debrief

25 min

Cosa non torna nel dataset

Da Excel a Python

In Excel il file lo apri e lo guardi.

In Python lo carichi in una variabile e gli fai domande.

Il file originale non viene mai modificato.

PYTHON

```
import pandas as pd

df = pd.read_excel(url)
```

OUTPUT

```
(nessun output: il file
ora è nella variabile df)
```

Le librerie: attrezzi già pronti



`pandas`

Tabelle: leggere, filtrare, riassumere



`matplotlib`

Grafici



`statsmodels`

Statistica ed econometria

PYTHON

```
import pandas as pd
```

`import` prende la libreria; **`as pd`** le dà un nome corto.

Il DataFrame

	id_impresa	regione	anno	addetti
0	IMP0001	Emilia-Romagna	2019	10
1	IMP0001	Emilia-Romagna	2020	10
2	IMP0001	Emilia-Romagna	2021	11
…	…	…	…	…

PYTHON

```
df["regione"]    # una colonna sola
```

- Una tabella, come un foglio Excel
- Ogni riga è un'osservazione: un'impresa in un anno
- Ogni colonna ha un nome e un tipo
- A sinistra l'indice: il numero della riga, parte da 0

Le liste

```python
regioni = ["Lombardia", "Lazio", "Sicilia"]

len(regioni)        # 3
regioni[0]          # "Lombardia"
regioni[-1]         # "Sicilia"
regioni.append("Puglia")
```

- Una sequenza di valori tra parentesi quadre
- Si conta da 0, come in tutto Python
- Lo slicing funziona come sul testo
- **Più colonne di una tabella? Una lista di nomi.**

Selezionare righe e colonne

```
df["regione"]
```

una colonna

```
df[["regione", "anno"]]
```

più colonne: una lista, doppie parentesi

```
df.iloc[0:5]
```

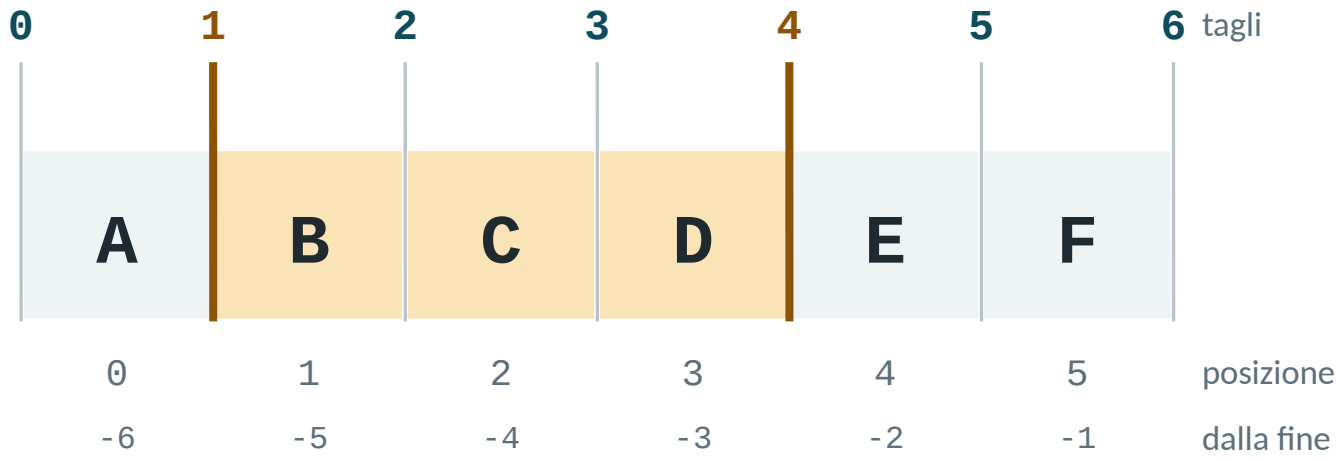
righe per posizione: 0, 1, 2, 3, 4

```
df[df["anno"] == 2020]
```

righe che rispettano una condizione: == chiede «è uguale?»

Attenzione in **`0:5`** il primo numero è incluso, il secondo escluso. E si parte da 0.

Slicing: dove cadono i tagli



`x[1:4]` → B C D: si taglia al 1 e al 4

`x[:3]` A B C

dall'inizio al taglio 3

`x[3:]` D E F

dal taglio 3 alla fine

`x[-2:]` E F

gli ultimi 2

`x[2]` C

un elemento solo

I numeri dello slicing non indicano le caselle, ma i tagli tra una casella e l'altra. Così fine − inizio = quanti elementi ottieni.

Omissioni e passo

Omettere inizio o fine

Se manca l'inizio si parte da 0; se manca la fine si arriva in fondo.

`x[:3]`	`= x[0:3]`	A B C
`x[3:]`	`= x[3:6]`	D E F
`x[:]`	`= x[0:6]`	tutto

Il terzo numero: il passo

[inizio:fine:passo] — ogni quante caselle prenderne una

`x[::2]`	A C E
`x[1::2]`	B D F
`x[::-1]`	F E D C B A

x = **A B C D E F** Con il passo negativo si va all'indietro: **`[::-1]`** inverte.

iloc è come un intervallo di Excel

	A	B	C	D
1	impresa	anno	addetti	regione
2	IMP0001	2019	10	Emilia-R.
3	IMP0001	2020	10	Emilia-R.
4	IMP0001	2021	11	Emilia-R.
5	IMP0001	2022	12	Emilia-R.
6	IMP0001	2023	11	Emilia-R.
7	IMP0002	2019	7	Lazio

Excel `A2:C6`

PYTHON

```
df.iloc[0:5, 0:3]
```

	Excel	iloc
Prima riga di dati	2 (la 1 è l'intestazione)	0
Fine dell'intervallo	inclusa (C6 c'è)	esclusa (5 non c'è)
Colonne	lettere: A, B, C	numeri: 0, 1, 2
Prima righe, poi colonne?	no: A2 = colonna, riga	sì: [righe, colonne]

Stesse celle: cambiano solo i numeri.

Fare domande a un oggetto

Metodo

un'azione: ha le parentesi

PYTHON

```
df.head(3)
```

OUTPUT

```
le prime 3 righe
```

Attributo

un'informazione: niente parentesi

PYTHON

```
df.shape
```

OUTPUT

```
(2515, 9)
```

df è un oggetto: dati, più tutte le cose che puoi chiedergli. Il punto le collega.

Come scoprire cosa sa fare un oggetto



`df. + Tab`

In Colab: l'elenco di tutto quello che puoi chiedere



`help(df.head)`

La spiegazione di un metodo, con esempi



La documentazione

pandas.pydata.org → API reference → DataFrame

Nessuno conosce tutti i metodi. Si cercano, ogni giorno, anche dopo anni.

Le domande da fare sempre a un dataset

1 **Quanto è grande?**

`df.shape`

2 **Di che tipo sono le colonne?**

`df.dtypes`

3 **Quanti valori diversi?**

`df["col"].nunique()`

4 **Quali valori, e quante volte?**

`df["col"].value_counts()`

Prima di qualunque analisi. Sempre.

Task: conosci il tuo dataset

Un collega ti manda imprese.xlsx. Ti puoi fidare?

60 minuti · apri il notebook dal README

Debrief

Cosa non torna nel dataset

I conti non tornano

2.500

righe attese

`500 imprese × 5 anni`

2.515

righe nel file

`df.shape`

15 righe in più.

Probabilmente duplicati: la stessa impresa, lo stesso anno, due volte.

Un controllo da fare a mente, in 10 secondi. Nessun codice lo fa per te.

Il fatturato è testo

PYTHON

```
df["fatturato"].mean()
```

ERRORE

```
TypeError: ... string ...
```

PYTHON

```
df["fatturato"].max()
```

OUTPUT

```
'n.d.'
```

La media si ferma con un errore. Il massimo no.

Il massimo di un testo è in ordine alfabetico: «n» viene dopo le cifre.

Perché è testo? Il file usa la virgola decimale italiana (1.032,9) e contiene "n.d.".

È la trappola "5000" * 2 della L1, su un file vero.

Lombardia non è lombardia

OUTPUT

```
Lombardia      302
...
lombardia       35
Lombardia       30
LOMBARDIA       30
Emilia-Romagna 147
Emilia Romagna  46
```

Per Python sono valori diversi: maiuscole, spazi finali, trattini.

Le due «Lombardia» uguali? Una ha uno spazio alla fine: non si vede, ma c'è.

Se raggruppi per regione così com'è, la Lombardia viene spezzata in quattro.

Cosa abbiamo trovato



Fatturato come testo

virgola decimale e "n.d."



15 righe in più

probabili duplicati



Regioni scritte male

maiuscole, spazi, trattini



Addetti negativi

-12: impossibile (bonus)

Solo uno di questi problemi ha prodotto un errore. Gli altri erano silenziosi.

Prossima volta: pulire i dati

Compito: Kaggle Learn – Pandas, prime due lezioni, ed esercizi C1-C2 del notebook